

Bug Report

Bug 1: Partial String Matching in Status Filter

Location / Endpoint

```
src/services/taskService.js -> getByStatus function  
GET /tasks?status=... endpoint
```

Expected Behavior

The status filter should only return tasks that exactly match the requested status string.

Actual Behavior

The service uses `.includes(status)` instead of exact string matching. For example, filtering by `do` will match both `done` and `todo`.

How It Was Discovered

A unit test in `taskService.test.js` created tasks with the status `done`. Calling `taskService.getByStatus('do')` unexpectedly returned the `done` tasks instead of returning an empty array.

Root Cause

```
const getByStatus = (status) => tasks.filter((t) => t.status.includes(status));
```

What a Fix Would Look Like

Change the filter to use strict equality:

```
const getByStatus = (status) => tasks.filter((t) => t.status === status);
```

Status

Fixed

Bug 2: Incorrect Pagination Offset Calculation

Location / Endpoint

```
src/services/taskService.js -> getPaginated function  
GET /tasks?page=1&limit=10 endpoint
```

Expected Behavior

Requesting page 1 should return the first batch of results (items 0 through `limit - 1`).

Actual Behavior

The offset is calculated as `page * limit`. For `page=1` and `limit=2`, the offset is `2`, which skips the first two items and effectively returns the second page instead of the first.

How It Was Discovered

A unit test generated 5 tasks and requested `page=1` and `limit=2`. It expected to receive Task 1 and Task 2, but received Task 3 and Task 4 instead.

Root Cause

```
const getPaginated = (page, limit) => {  
  const offset = page * limit;  
  return tasks.slice(offset, offset + limit);  
};
```

What a Fix Would Look Like

Adjust the calculation to treat page 1 as the first page:

```
const offset = (page > 0 ? page - 1 : 0) * limit;
```

Status

Fixed

Bug 3: Cannot Combine Status Filtering and Pagination

Location / Endpoint

`src/routes/tasks.js` -> `GET /tasks` endpoint

Expected Behavior

Requesting `GET /tasks?status=todo&page=1&limit=10` should return a paginated list of tasks that have the `todo` status.

Actual Behavior

The routing logic checks if `status` is provided and returns early with the unfiltered list of tasks matching the status, completely ignoring the `page` and `limit` query parameters.

How It Was Discovered

An integration test requested `?status=todo&page=1&limit=1` expecting 1 item back, but it received all `todo` items because pagination was skipped.

Root Cause

```
if (status) {  
  const tasks = taskService.getByStatus(status);  
  return res.json(tasks); // Returns early!  
}
```

What a Fix Would Look Like

Modify the logic (or the service method) to apply the status filter first, and then paginate the filtered result, returning the combined array.

Status

Fixed

Bug 4: Completing a Task Resets its Priority

Location / Endpoint

`src/services/taskService.js` -> `completeTask` function
`PATCH /tasks/:id/complete` endpoint

Expected Behavior

Marking a task as complete should only update its `status` to `done` and set the `completedAt` timestamp. Other properties should remain unchanged.

Actual Behavior

The function hardcodes `priority: 'medium'` into the updated task, accidentally overwriting the task's existing priority.

How It Was Discovered

A test created a task with a `high` priority and then called `completeTask`. Afterwards, the task's priority had unexpectedly changed to `medium`.

Root Cause

```
const updated = {
  ...task,
  priority: 'medium', // Unnecessary and destructive overwrite
  status: 'done',
  completedAt: new Date().toISOString(),
};
```

What a Fix Would Look Like

Remove the `priority: 'medium'` line from the `updated` object in `completeTask`.

Status

Fixed

Bug 5: Update Endpoint Allows Overwriting System-Managed Fields

Location / Endpoint

```
src/services/taskService.js -> update function
PUT /tasks/:id endpoint
```

Expected Behavior

Clients should not be able to change internal, system-managed fields such as `id` or `createdAt`.

Actual Behavior

The `update` function blindly merges the incoming `fields` object with the existing task. If a client includes an `id` or `createdAt` field in the body, it will overwrite the existing internal values.

How It Was Discovered

A test explicitly passed `{ id: 'hacked-id', createdAt: '1999-01-01' }` to the `taskService.update()` function and successfully changed the task's ID and creation timestamp.

Root Cause

```
const updated = { ...tasks[index], ...fields };
```

What a Fix Would Look Like

Filter out protected fields before merging, for example:

```
const { id: __, createdAt: __, ...safeFields } = fields;
const updated = { ...tasks[index], ...safeFields };
```

Status

Fixed

Bug 6: Empty Strings Bypass `dueDate` Validation

Location / Endpoint

`src/utils/validators.js` -> `validateCreateTask` and `validateUpdateTask`
`POST /tasks` and `PUT /tasks/:id` endpoints

Expected Behavior

A `dueDate` should be either omitted, `null`, or a valid ISO string. Passing an empty string `""` should trigger a validation error.

Actual Behavior

The validation uses a logical AND (`body.dueDate && ...`), which short-circuits if the value is falsy. An empty string evaluates to false, so the invalid date string validation is entirely skipped and the empty string is accepted.

How It Was Discovered

A validation unit test checked if `{ dueDate: "" }` would throw an error. It did not, and returned `null` (success) instead.

Root Cause

```
if (body.dueDate && isNaN(Date.parse(body.dueDate))) {
```

What a Fix Would Look Like

Explicitly ensure the value is defined and not null before parsing:

```
if (body.dueDate !== undefined && body.dueDate !== null) {  
  if (body.dueDate === '' || isNaN(Date.parse(body.dueDate))) {  
    return 'dueDate must be a valid ISO date string';  
  }  
}
```

Status

Fixed

Observations / Questions

1. Inconsistent Status Names in Documentation

The `README.md` claims the available statuses are `pending` | `in-progress` | `completed`. However, `ASSIGNMENT.md` and the actual codebase (in `validators.js`) use `todo` | `in_progress` | `done`. This inconsistency should be updated in the README.

2. PUT vs PATCH

The API defines `PUT /tasks/:id` for updating a task. A `PUT` request generally implies a full replacement of the resource, but the current `validateUpdateTask` and `update` logic allows partial updates (fields can be omitted). This behaves more like a `PATCH`. A question for the team would be whether we should enforce full-replacement for

PUT or rename the endpoint to PATCH .

Solutions

1. Bugs We Fixed

Bug 1 — Partial String Matching in Status Filter

Problem:

The `GET /tasks?status=...` query parameter could match multiple statuses if one was a substring of another (e.g., `status=do` would match both `done` and `todo`).

Root Cause:

The existing implementation in `taskService.js` used `tasks.filter((t) => t.status.includes(status))` instead of strict equality.

Fix:

Changed the filter to use strict string equality: `tasks.filter((t) => t.status === status)` .

Approach:

Strict equality ensures that we only return tasks that exactly match the requested status string, adhering to standard REST API querying expectations.

Tests:

Verified by `should not return tasks with partially matching status strings` in `tests/taskService.test.js` .

Bug 2 — Incorrect Pagination Offset

Problem:

Requesting page 1 (`page=1&limit=2`) skipped the first two items and effectively returned the second page instead.

Root Cause:

The offset in `taskService.getPaginated()` was calculated as `page * limit` . For page 1, the offset was set to `limit` , meaning the slice began at index `limit` rather than index `0` .

Fix:

Changed the offset calculation to `const offset = Math.max(0, page - 1) * limit;` .

Approach:

This treats page 1 as the first logical page (index 0) while preventing negative offsets. It conforms to standard pagination implementations.

Tests:

Verified by `should return the correct first page (page 1)` and `should return correct subsequent pages` in `tests/taskService.test.js` and `should return paginated tasks correctly` in `tests/tasks.routes.test.js`.

Bug 3 — Status Filtering + Pagination Cannot Be Combined

Problem:

Passing both `status` and `page / limit` in the URL resulted in the `page` and `limit` being completely ignored, returning the entire list of tasks matching the status.

Root Cause:

In `routes/tasks.js`, the route handler returned early if `status` was present, bypassing the pagination logic entirely.

Fix:

Updated `taskService.getPaginated(page, limit, status = null)` to accept an optional `status` parameter so it can filter the items before paginating. Updated the route handler in `routes/tasks.js` to combine these query strings properly.

Approach:

Combining filter logic directly within the `getPaginated` service function is clean and avoids duplicating data manipulation logic inside the Express route. It returns early only if filtering/paginating is needed.

Tests:

Verified by `should allow combining status and pagination` in `tests/tasks.routes.test.js`.

Bug 4 — Completing a Task Resets Priority

Problem:

Marking a task as complete via `PATCH /tasks/:id/complete` reset the task's priority to `medium`, even if it was originally `high` or `low`.

Root Cause:

The `completeTask` function in `taskService.js` explicitly hardcoded `priority: 'medium'` into the updated object when saving.

Fix:

Removed the `priority: 'medium'` line from the `updated` object definition.

Approach:

A complete action should only mutate completion-related state (`status` and `completedAt`). Preserving existing properties prevents accidental data destruction.

Tests:

Verified by `should not overwrite the task priority` in `tests/taskService.test.js`.

Bug 5 — Update Allows Overwriting `id` and `createdAt`

Problem:

Clients were able to maliciously or accidentally change a task's internal `id` and `createdAt` fields via `PUT /tasks/:id`.

Root Cause:

The `update` function in `taskService.js` blindly merged the provided `fields` object directly onto the existing task using object spread `{ ...tasks[index], ...fields }`.

Fix:

Deconstructed `id` and `createdAt` out of `fields` to create a `safeFields` object, and only merged `safeFields`.

Approach:

Sanitizing the input object guarantees that system-managed immutable properties cannot be tampered with by clients, protecting internal data integrity.

Tests:

Verified by `should not allow overwriting the id or createdAt fields` in `tests/taskService.test.js`.

Bug 6 — Empty `dueDate` Bypasses Validation

Problem:

Passing an empty string `""` as a `dueDate` during creation or updates was silently accepted, skipping the ISO date validation.

Root Cause:

The validation condition `if (body.dueDate && isNaN(Date.parse(body.dueDate)))` short-circuited because empty strings are falsy in JavaScript.

Fix:

Changed the check to `if (body.dueDate !== undefined && body.dueDate !== null && isNaN(Date.parse(body.dueDate)))` in both `validateCreateTask` and `validateUpdateTask`.

Approach:

Explicitly validating that the value is neither `undefined` nor `null` ensures that falsy values like empty strings are accurately evaluated by `Date.parse()`, leading to proper rejection.

Tests:

Verified by `should return error if dueDate is an empty string` in `tests/validators.test.js`.

2. New Feature — PATCH `/tasks/:id/assign`

Implementation

The new `PATCH /tasks/:id/assign` endpoint allows clients to assign a task to a user. This functionality was implemented by adhering strictly to the project's separation of concerns:

1. **Validation Layer:** Added `validateAssignTask` to `src/utils/validators.js` to ensure incoming requests contain a valid assignee.
2. **Service Layer:** Added an `assignTask` function to `src/services/taskService.js` to update the task in the data store.
3. **Route Layer:** Added the `PATCH` route to `src/routes/tasks.js` to parse the request, validate the payload, invoke the service, and handle HTTP responses.

Request

The endpoint expects a `PATCH` request to `/tasks/:id/assign` with a JSON payload containing an `assignee` key:

```
{
  "assignee": "string"
}
```

Response

- **Success (200 OK):** Returns the updated task object containing the new `assignee` property.
- **Not Found (404 Not Found):** Returns `{ "error": "Task not found" }` if the task ID does not exist.
- **Bad Request (400 Bad Request):** Returns a validation error if the request body is missing or malformed.

Validation

The `validateAssignTask` function requires `assignee` to be present and to be a non-empty string. It rejects payloads that omit the field, pass empty or whitespace-only strings, or pass non-string types.

Error Handling

- Handled within the route: If validation fails, it returns a 400 Bad Request. If the service fails to find the task, it returns a 404 Not Found.

Edge Cases

- **Already Assigned Task:** Reassigning a task is allowed. It gracefully overwrites the previous assignee string, which is consistent with typical task management behavior.
- **Empty String:** Banned. If someone attempts to assign to an empty string, a validation error is thrown. (Typically, unassigning might be handled explicitly by passing `null`, but the assignment specification requested a 'string', so the logic ensures it's a substantive string).

Design Decisions

- Followed the existing architecture exactly by separating validation, business logic, and routing.
- The `taskService.assignTask` method clones and overwrites the task using `findIndex`, similar to how `update` and `completeTask` are structured.

Tests

Added exhaustive unit and integration tests:

- `validators.test.js` : Unit tests validating missing, empty, and invalid `assignee` inputs.
- `taskService.test.js` : Unit tests verifying `assignTask` correctly applies the assignee and allows subsequent reassignments, returning `null` for missing tasks.
- `tasks.routes.test.js` : Integration tests verifying successful assignments, 404 behavior, and 400 behavior when missing/invalid bodies are submitted. All edge cases verified against HTTP contract.

3. Submission Notes

What I'd Test Next

If I had more time, I would add more edge-case and integration tests, including concurrent task updates, unusual input values, and additional combinations of filtering and pagination. I would also test the API under higher request volumes to identify any performance or reliability issues.

What Surprised Me

One thing that surprised me was that several existing behaviors had subtle issues that were not immediately obvious from the API surface, particularly the interaction between status filtering and pagination. The API also had validation and update behaviors that could lead to unexpected task data changes.

Questions Before Shipping to Production

Before shipping this API to production, I would clarify:

- What authentication and authorization requirements are expected?
- Should the in-memory task store be replaced with a persistent database?
- What are the expected API response and error formats?
- What are the requirements for logging, monitoring, and rate limiting?
- What should be the expected behavior when multiple users update the same task concurrently?